TryHackMe Document: Fools Mate - CTF Writeup

This report provides a detailed walkthrough and technical analysis for solving the **TryHackMe - Fools Mate** challenge. The challenge involves analyzing a web-based chess application ("Endgame"), inspecting front-end JavaScript logic, and manipulating client-side state variables to bypass security restrictions and capture the flag.

Challenge Overview

Attribute	Details
Platform	TryHackMe
Target Application	Endgame (Web-Based Chess Game)
Vulnerability Class	Insecure Client-Side Logic & State Manipulation
Tools Used	Browser Developer Tools (Inspect Mode / JavaScript Debugger)

Methodology & Execution

1. Initial Observation & Behavior Analysis

Upon navigating to the target web application, **Endgame**, an interactive chess environment is presented. During gameplay analysis:

- Delivering checkmate to the enemy king (executing the move Rook to a8) triggers a client-side execution lock.
- The game halts gameplay execution and displays an alert warning/threatening a device shutdown.

2. Source Code Inspection & Debugging

To investigate the mechanism behind the lock and threat, Browser Developer Tools (Inspect Mode) were opened to analyze the application's JavaScript assets:

1. Located the primary script file: app.js.

2. Identified the threat/shutdown function responsible for interrupting gameplay at **line 257**.
3. Configured breakpoints and debugger options within the Developer Tools to control execution flow when line 257 is reached.

3. Variable Manipulation & Flag Extraction

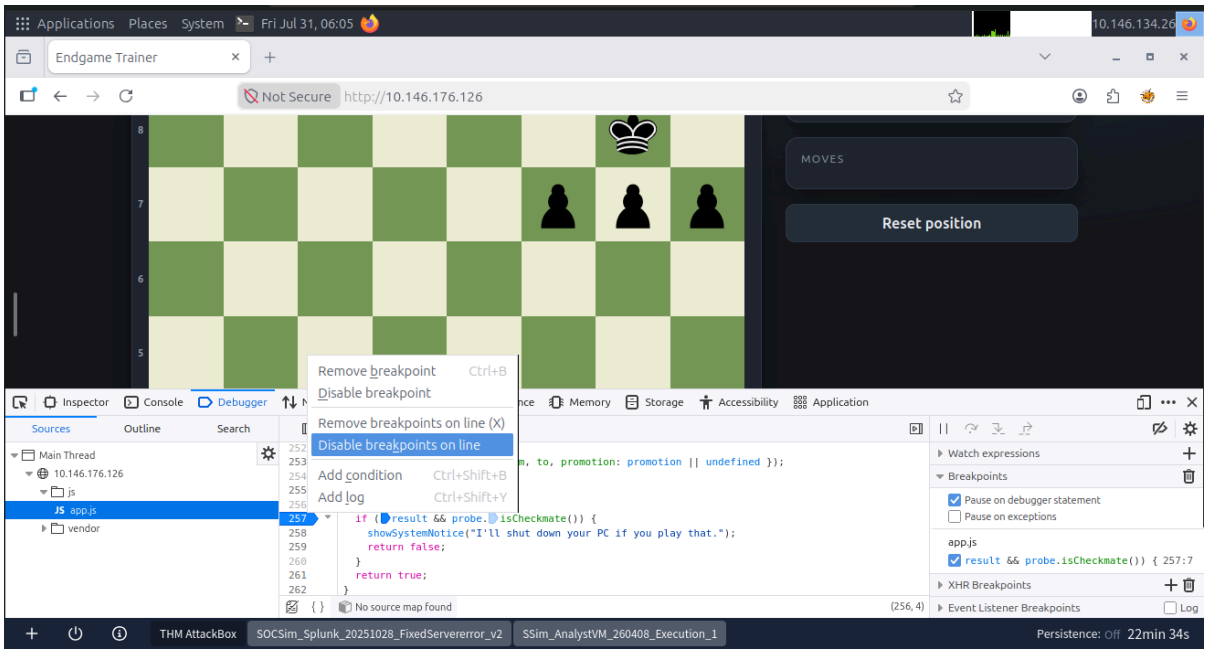
By taking advantage of the active debugging pause during the checkmate move execution:

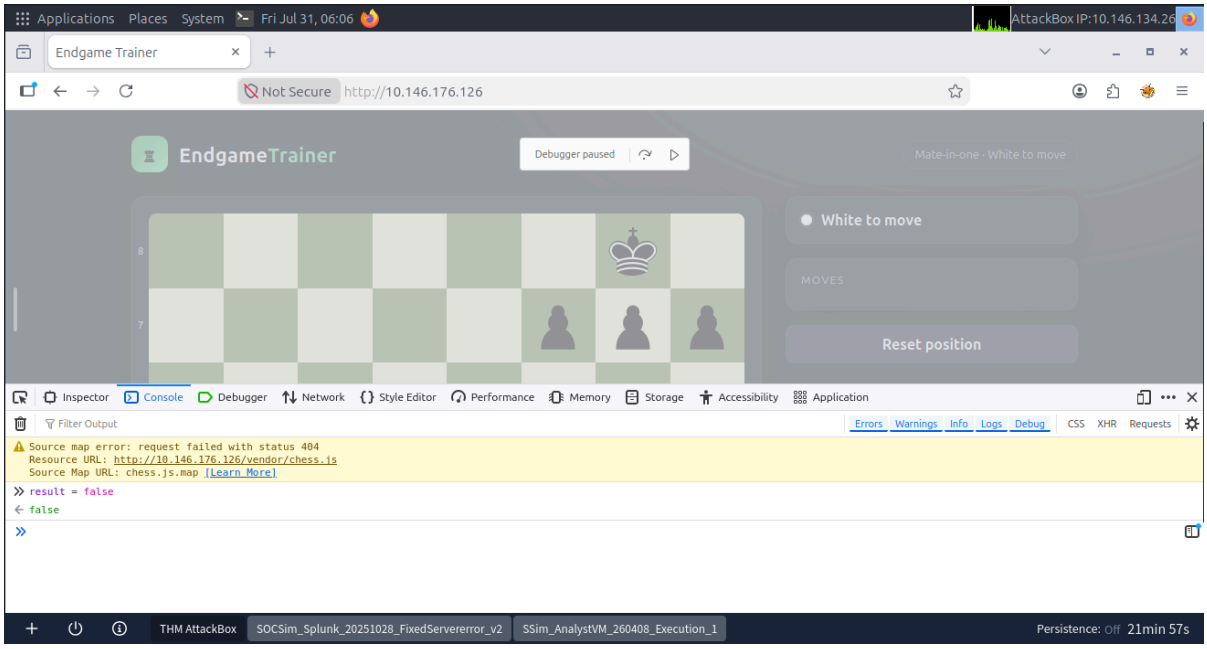
- When the checkmate occurred and the debugger hit the designated line, execution paused automatically.
- Using the Developer Console, the boolean condition controlling the defensive routine was overwritten by executing:
  `result = false;`
- Resuming script execution allowed the altered state to bypass the shutdown handler and render the flag successfully.

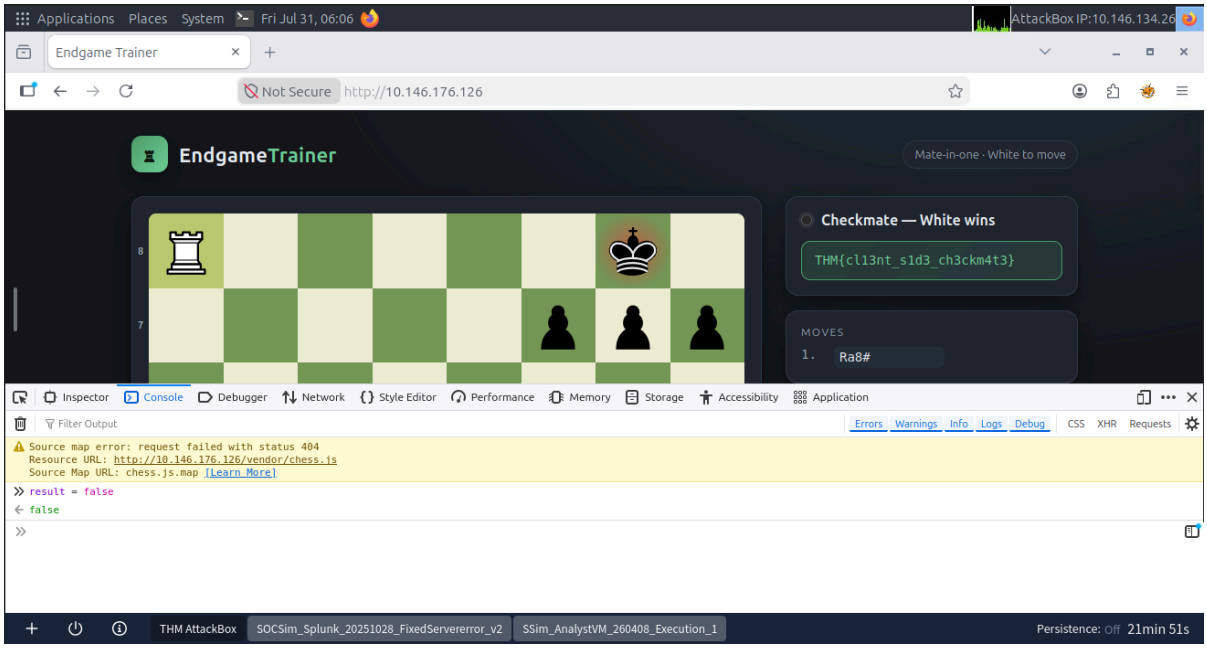
Conclusion & Remediation

This challenge demonstrates the risk of relying purely on client-side controls for critical logic or flag verification. Because JavaScript runs directly within the user's browser, developers cannot trust client-side state or variables to enforce security mechanisms.

Images




Applications Places System Fri Jul 31, 06:06
AttackBox IP:10.146.134.26
Endgame Trainer
Not Secure http://10.146.176.126
EndgameTrainer
Debugger paused
Mate-in-one · White to move
White to move
MOVES
Reset position
Inspector Console Debugger Network Style Editor Performance Memory Storage Accessibility Application
Filter Output
Errors Warnings Info Logs Debug CSS XHR Requests
Source map error: request failed with status 404
Resource URL: http://10.146.176.126/vendor/chess.js
Source Map URL: chess.js.map [Learn More]
result = false
false
THM AttackBox
SOCSim_Splunk_20251028_FixedServererror_v2
SSim_AnalystVM_260408_Execution_1
Persistence: Off 21min 57s


Applications Places System Fri Jul 31, 06:06
AttackBox IP:10.146.134.26
Endgame Trainer
Not Secure http://10.146.176.126
EndgameTrainer
Mate-in-one · White to move
Checkmate — White wins
THM{cl13nt_s1d3_ch3ckm4t3}
MOVES
1. Ra8#
Inspector Console Debugger Network Style Editor Performance Memory Storage Accessibility Application
Filter Output
Errors Warnings Info Logs Debug CSS XHR Requests
Source map error: request failed with status 404
Resource URL: http://10.146.176.126/vendor/chess.js
Source Map URL: chess.js.map [Learn More]
result = false
false
THM AttackBox
SOCSim_Splunk_20251028_FixedServererror_v2
SSim_AnalystVM_260408_Execution_1
Persistence: Off 21min 51s